When Do Parameter Budgets Misprice CPU Inference? A Matched-Budget Test of Byte-Traffic Pruning Allocation

Prabhu Rangaraju
Independent Researcher

Abstract

Pruning budgets for language models are almost always stated in parameter count, yet CPU decode cost is plausibly driven by the bytes moved per token: weight bytes streamed each step plus KV-cache bytes read at the occupied context. We test, under a pre-registered protocol, whether allocating a $\sim$25% structured prune of Llama-3.2-3B by an analytical per-token byte-traffic model beats allocating it by parameter count, at fixed CPU thread caps (2 and 4 threads, both pinned to 2 cores). Seven allocations search one shared unit space (layers, FFN channels, KV groups) and differ only in the cost model used to choose the cut. The result is a bounded negative. In a held-out (leave-one-variant-out) prediction contest, the traffic model does not beat the parameter model at either thread cap, and the predeclared byte allocation is not reliably faster than the parameter allocation to the registered standard; the traffic model's own predicted speedups are systematically optimistic. Within this matched-budget CPU pruning family, an analytical weight-plus-KV traffic model therefore did not improve held-out prediction or registered selection over parameter count. The negative is about the analytical model, not about traffic-aware pruning as such: pruning by *measured* latency remains untouched by our result. Two scope limits bind the result: we measure decode speed only (pruned-model quality is not evaluated), and all timing is on a shared 2-core virtual machine whose tenant-load variance — including a documented load-era effect on the earliest cells — is a formal limitation. Descriptively, the byte-greedy pick's mean selection regret is low — a good pick, Section 4.3 argues, that the model itself does not explain — and a post-hoc sensitivity excluding the saturated longest context reverses the 4-thread contest, localising the failure there; the frozen verdict is unchanged. We report the mechanisms consistent with the failure, so the boundary of the traffic model's usefulness is itself the contribution.

1 Introduction

Structured pruning removes whole layers, channels, or attention components so that a smaller model runs faster on real hardware. The budget for such a prune is conventionally a parameter count: remove $x\%$ of the weights. Parameter count is, however, a proxy. At batch size 1, autoregressive decode is widely treated as memory-bound: each step must stream the model's weights once and read the accumulated key–value (KV) cache, whose size grows with context. A cost model based on per-token byte traffic — weight bytes streamed per step plus KV bytes read at the occupied context — is therefore an attractive alternative: it prices KV-heavy allocations differently from parameter-heavy ones, and it predicts that two allocations with identical parameter counts can have different decode costs, especially at long contexts.

This intuition has been used to *analyse* inference [Bi et al., 2026] and named as an untested extension by pruning work that itself budgets in parameters [Wang et al., 2026], but we find no prior controlled test of the allocation question (literature search, October 2026): *within one matched-budget pruning family, does choosing the cut by byte traffic produce a faster model, and does the traffic model predict held-out allocations' speed better than parameter count?*

We answer with a pre-registered, single-model, single-runtime experiment designed so that the two cost models are the only thing that differs between arms. Our findings are negative in a specific, bounded sense:

1. **Prediction.** On held-out variants, the traffic model does not out-predict the parameter model (Section 4.2).

2. **Registered selection.** The predeclared byte allocation fails the pre-registered $\geq$10% paired-advantage criterion at both thread caps (Section 4.1).

3. **Model calibration.** The traffic model predicts 23–33% advantages at cap 2 where 1–8% are observed, so the failure is in the model, not only in the decision rule.

We also report selection regret descriptively (Section 4.3): the byte-greedy pick was in raw terms a decent pick, which makes the model's inaccuracy more, not less, informative — a good outcome obtained for the wrong stated reason is not evidence for the cost model.

2 Related work

Measured-latency pruning. ZipLM prunes by target-runtime latency lookup tables rather than by size [ZipLM, 2023]. HAPE ranks structures by measured on-device latency differences for Llama-2-7B on a Xeon CPU, the closest deployment precedent to ours [Zhao et al., 2025]. Both bypass analytical cost models entirely; neither tests whether an analytical traffic model adds predictive value over parameters on unseen allocations. **Joint FFN–KV allocation.** GRASPrune allocates a single budget jointly over FFN channels and KV groups, but states that "we use parameter count as the budget proxy"; a bandwidth-aware cost is proposed in its §3.1 and named as a limitation, not evaluated [Wang et al., 2026]. Our experiment is, to our knowledge, the controlled test of that stated extension. **KV-targeted pruning.** KVPruner prunes KV channels and reports that equal parameter counts can yield different speeds, but its inference-overhead evaluation disables caching, so its numbers do not speak to cached decode [KVPruner, 2024]. **Traffic and roofline models.** RooflineBench defines per-token traffic as weight bytes plus active KV reads/writes — the formula family we use — and benchmarks it across on-device hardware including Llama-3.2-1B, without a pruning-allocation test [Bi et al., 2026]. Analytical inference models separate weight and KV memory costs [Pope et al., 2023]; roofline surveys argue low-batch decode is memory-bound [Yuan et al., 2024]; CPU characterisation work shows decode performance depends on memory hierarchy, placement, and core configuration in ways a single bandwidth constant cannot capture [CPU Inference Study, 2024]; and kernel-level simulation finds roofline predictors retain large errors unless kernel efficiency is learned [Yao et al., 2026]. Our design asks whether the simplest analytical traffic model earns its keep for one concrete decision: where to cut.

3 Method

3.1 Variants and cost models

We prune Llama-3.2-3B-Instruct (GGUF Q4_K_M; KV cache fp16) by $\sim$25% of parameters. All seven variants are chosen from one shared search space (whole layers, FFN channels, KV groups); allocations differ only in the cost model that prices each unit: parameter count, marginal byte traffic ΔB at the design contexts, or the resulting mix. Final parameter counts span 2.398–2.442B (1.84% spread); parameter count enters all analyses as a covariate, so exact equality is not load-bearing. SIMD-alignment constraints bind all variants (FFN multiples of 256, widths multiples of 64, head dimension 128).

Both candidates are produced by applying their cost model to the *identical* candidate space, so their differing composition is the treatment, not a confound: the parameter model spends the whole budget on FFN width because layers and KV heads are expensive in parameters per unit, while the byte model spends it on layers and KV heads because they are expensive in traffic per unit. The remaining five variants span intermediate selections of the same space, so the prediction contest (Section 4.2) is over allocations, not only over the two endpoints. Concretely, V-width keeps all 28 layers and 8 KV heads per layer and trims the FFN intermediate size uniformly to 5888; V-mixed removes layers 18 and 20–23, keeps 4 KV heads per layer, and trims the FFN intermediate size uniformly to 8192.

Variant	Layers	$\sum$ KV-heads	Params (B)	Allocation rule
V-width	28	224	2.442	parameter count (FFN tri
V-depth	21	168	2.409	7 whole layers dropped
V-depth-lite	24	192	2.414	4 whole layers dropped
V-depth-alt	21	168	2.409	7 layers, alternate positio
V-KV	28	112	2.398	KV heads halved (GQA
V-KV-lite	28	168	2.442	KV heads 8→6 (export i
V-mixed	23	92	2.420	predeclared byte-greedy

Table 1: The seven allocations. V-width is the parameter-count candidate and V-mixed, predeclared before measurement, is the byte-traffic candidate for the head-to-head contrast. Parameter counts verify the matched budget (2.398–2.442B, 1.84% spread); tensor-payload bytes per variant are 1.526–1.552 GB.

The traffic model prices a decode step at occupied context c as $B(c) = B_{\text{weights}} + B_{\text{KV}}(c)$, with $B_{\text{KV}}(c) = 2 \times (\sum \text{KV-heads}) \times 128 \times 2\,\text{B} \times c$. For the unpruned base this gives KV shares of total traffic of $\approx$24% at 4k, 39% at 8k, and 56% at 16k context — the long-context regime is where a traffic model has room to differ from parameter count, so contexts below 4096 are reported but the registered prediction contest uses $C \geq 4096$.

3.2 Protocol

The protocol, criteria, and analysis definitions were preregistered before any stage-1 measurement (study record SPEC v0.3, 2026-10-23, frozen before any stage-1 run; runtime amendments were logged in the same record as they occurred and are summarised where they bear on the results). Timing uses llama.cpp (`llama-server`) on a 2-core virtual machine (the guest presents 2 vCPUs of an AMD EPYC 9D25 host). Thread caps are llama.cpp thread counts ($-t$ 2 and $-t$ 4) under a `taskset -c 0,1` pin to two cores; "cap 4" is four threads on two cores — an oversubscribed cap, not a four-core result — and is labelled as such throughout. Each measurement cell (variant $\times$ cap $\times$ context $\in \{512, 2048, 4096, 6144, 8192, 16384\}$) runs in a fresh server process ("virgin" protocol, adopted after a

slot-history contamination artifact was caught and its 30 pre-protocol reps excluded), with prefill-only primes, 5 warmup and 5 recorded repetitions of exactly 128 decode steps on fixed token IDs. All decoding is single-sequence (batch size 1). The primary metric is CPU seconds per token; per-rep host load and involuntary context switches are recorded. In total 521 virgin reps are banked, of which 451 belong to the six primary variants (see Section 4.5); the registered contest fits use the $C \geq 4096$ subsets (135 reps at cap 2, 136 at cap 4).

3.3 Pre-registered criteria

Three frozen tests decide the claim. (i) *Prediction contest:* regressions of log CPU-s/token on either parameter count or modeled byte traffic (each with a layer-count term), judged on leave-one-variant-out (LOVO) relative error over $C \geq 4096$ reps; the traffic model wins only if it has lower LOVO error *and* ΔAIC (parameters $-$ traffic) > 10. LOVO relative error is the mean, over the held-out variant's reps, of $|\text{pred} - \text{obs}|/\text{obs}$; because every held-out variant is an extrapolation to an unseen architecture, absolute values are large for both models and the contest is comparative. (ii) *Registered hurdle:* in genuine temporally-paired blocks (five blocks per cap $\times$ {8192, 16384}, adjacent fresh-process cells, randomised order), the upper 95% confidence bound of the paired log-time ratio V-mixed/V-width must be < 0.90 at both contexts per cap. (iii) *Selection regret* (post-hoc and non-registered; descriptive only, computed at manuscript stage from banked data): for each cell, a selector's regret is its mean time minus the fastest primary variant's mean, relative to the fastest mean.

4 Results

4.1 Registered hurdle: not met

Cap	Ctx	Paired ratio	95% CI	Traffic-predicted
2	8192	0.917	[0.824, 1.020]	0.771
2	16384	0.986	[0.934, 1.041]	0.673
4	8192	0.818	[0.795, 0.842]	0.771
4	16384	0.817	[0.609, 1.096]	0.673

Table 2: Paired V-mixed/V-width time ratios (five retained pairs per cell, from the paired-block manifest; three restart-split blocks were replaced whole). The traffic model's predicted ratio is cap-independent (it prices bytes, not threads). The hurdle (upper bound < 0.90 at both contexts) is met in no cap: cap 2 fails on effect size, cap 4 passes at 8192 but is too imprecise at 16384.

The byte candidate is faster than the parameter candidate at 4 threads ($\approx$0.82) but not at 2 threads (0.92 / 0.99, CIs include 1), and nowhere to the registered standard. The traffic model's predicted ratios are systematically optimistic.

4.2 Prediction contest: traffic does not win

Cap	Params LOVO err	Traffic LOVO err	ΔAIC (P$-$B)
2	0.743	0.689	$+2.21$ (needed > 10)
4	0.284	0.538	-19.11 (params win)

Table 3: Held-out prediction, $C \geq 4096$ (135 reps at cap 2, 136 at cap 4). Including the invalid variant as a sensitivity gives the same outcome (ΔAIC $+3.37$ / -19.56).

At 2 threads the traffic model's small LOVO edge is far below the registered ΔAIC margin; at 4 threads the parameter model is better on both criteria. The unscaled raw-byte predictor is far worse (held-out relative error 4.34 / 4.97), so the contest is between the fitted models, and the fitted traffic model still does not earn its extra structure. With only six valid variants, these point estimates carry wide uncertainty, which we quantify by a cluster bootstrap over variants (2,000 resamples, identical definitions): cap 2 ΔAIC median -0.91, 95% interval $[-28.8, +37.3]$, $P(\Delta\text{AIC} > 10) = 0.16$; cap 4 median -8.90, interval $[-138.4, +7.29]$, $P(> 10) = 0.02$. The verdict is therefore a criteria verdict, not a significance claim: at cap 2 the traffic model is simply not shown to be better by anything like the registered margin, and at cap 4 the point estimates and the bootstrap both favour parameters. A post-hoc sensitivity (requested in review; not part of the frozen criteria) excludes the saturated 16384 context, refitting the identical contest on $C \in \{4096, 6144, 8192\}$ (98 / 95 reps; the pipeline first replicates the frozen numbers exactly). The cap-2 outcome is unchanged (ΔAIC $+3.70$, still below the > 10 margin), but the cap-4 outcome reverses: traffic LOVO 0.207 vs params 0.273, ΔAIC $+20.27$ — the traffic model wins there. The frozen verdict is unaffected, but the sensitivity localises the failure: it is concentrated at the longest context, where saturation and contention are worst, not uniform across the regime; whether that concentration is a true context-scaling effect or contention noise at 16384 cannot be separated on the banked data.

4.3 Selection regret

Regret against the measured fastest primary variant, per cell (frozen cell means): V-mixed's regret is 0% at 4096–6144 (both caps) and at cap-4 8192, rising to 9.4% (cap 2, 8192, fastest V-KV) and 5.5–5.8% at 16384 (fastest V-depth-alt). V-width's regret is 6.6–47.0%. Mean regret over $C \geq 4096$ is 3.8% (V-mixed) vs 24.4% (V-width) at cap 2 and 1.4% vs 21.3% at cap 4. Two cautions bind this descriptive measure: at cap-2 8192 the cell mean for V-width is variance-

inflated (the paired ratio, 0.917, is authoritative for the two-candidate comparison — excluding that one cell, mean regret is 1.9% (V-mixed) vs 16.9% (V-width) at cap 2, so the ordering does not depend on it), and regret rewards the pick, not the model — V-mixed also removes five layers, so its speed is not attributable to its traffic advantage alone. The registered verdict rests on Sections 4.1–4.2; regret shows the practical stakes were asymmetric: following the traffic model would not have hurt *here*, but the model gives no reliable reason to expect that elsewhere, since it cannot predict held-out allocations or its own margin.

4.4 Diagnostics and saturation

Fresh diagnostic remeasurement of the earliest pool cells did not reproduce their original slowness: those cells were measured in the run's highest-contention window (host load 8–10 from other tenants), a load-era effect we carry as a caveat (it biases, if anything, against the depth allocations, not against the parameter model; the reference V-width agrees across eras at 6144 and runs 0.79–0.88$\times$ at 16384). At 16384, V-depth-alt and V-mixed saturate at cap 4 (wall tok/s falls vs cap 2); their cap comparisons at 16384 are saturation-regime and are not read as clean cap effects. On load sensitivity: the head-to-head contrast is load-matched by construction (temporally adjacent paired blocks, §4.1), the diagnostics bound the load-era effect directly, and no post-hoc covariate adjustment is applied because none was pre-registered — the pre-registered design choice is reported as a limitation (Section 6) rather than repaired after the fact.

4.5 The invalid variant

A teacher-forced logits check (144 positions) found V-KV-lite's GGUF export functionally invalid (top-1 agreement 0.000 vs the base reference's 0.958; a converter tensor-layout bug, not a quantisation effect). Per the frozen rule it is excluded from the primary fit (its timings are retained and the including-fit is reported as sensitivity, with the same contest outcome). Its timings are not evidence for a functionally valid candidate.

5 Discussion

One distinction organises everything below: *selection regret is not model accuracy.* Section 4.3 shows the byte-greedy pick was, in hindsight, a low-regret pick; this section is about why that does not rehabilitate the traffic model. A model earns its keep by predicting — held-out allocations (it did not, Section 4.2) and its own margins (it over-predicted them, Section 4.1). Nor should the parameter model be oversold in the other direction: absolute LOVO errors are large for *both* models (0.28–0.74 across caps), so the accurate summary is that the traffic model adds no reliable advantage over a weak baseline, not that parameter count predicts well. A good pick justified by an inaccurate model is luck with a plausible story, and the next allocation decision made with the same model has no such guarantee. Four mechanisms are consistent with the failure, and we present them as explanations to test, not as rescues. **(1) Parameters already price most of the traffic.** At fixed precision, weight bytes are a constant multiple of the weights streamed, and weights dominate traffic except at the longest contexts; the KV term — the traffic model's only independent signal — varies across our allocations less than the allocations' other properties do. Even at 16384, where KV is $\approx$56% of modeled base traffic, the predicted advantage (0.673) missed the observed ratio (0.986) by a wide margin, so signal size alone does not explain the miss. **(2) Logical bytes are not measured traffic** — in our judgement the primary driver of the model's inaccuracy. Cache residency, tensor layout, and operator fusion change actual memory movement; an inventory of tensor bytes sees none of these [Zhao et al., 2025]. Every other mechanism below operates on top of this gap between the traffic the model counts and the traffic the hardware performs. **(3) Kernel efficiency is shape-dependent.** Removing channels or heads changes matrix shapes and attained efficiency, not just work volume; this is why measured-latency methods price whole configurations [ZipLM, 2023] and why roofline predictors need learned efficiency terms [Yao et al., 2026]. **(4) Thread saturation on a small, oversubscribed pin.** With 4 threads on 2 pinned cores, scheduling and contention — including other tenants' load on this shared machine — move times by more than the modeled traffic differences in several cells. The evidence for saturation itself is narrow (two variants at 16384, §4.4); we offer it as a contributing mechanism, not a demonstrated primary cause. The practical guidance is narrow: for a one-off pruning decision on a fixed runtime, measuring candidate latencies directly (as HAPE and ZipLM do) dominates both analytical models; between the two analytical models, parameter count is the cheaper and, here, no worse predictor.

6 Limitations

This is a single model family (Llama-3.2-3B), a single runtime (llama.cpp, Q4_K_M, fp16 KV), a single shared 2-core virtual machine with substantial tenant load variance (per-rep load is recorded; no covariate adjustment was pre-registered), and two thread caps on that pin. The 2-core pin is itself a scope limit: it may create an artificial bottleneck that masks traffic-model benefits a larger multicore setup would show, and our negative result should not be read as covering that regime. The full pre-registered specification and its amendment record are retained with the study records (Data availability). We measure decode speed only: model quality after pruning is not evaluated, so no speed–quality

frontier is claimed. One of seven variants was export-invalid and excluded from the primary fit. The early-pool load-era caveat (Section 4.4) applies to the depth allocations' original cells; its direction is, if anything, against the depth allocations — the parameter-side reference variant agrees across measurement eras, so the caveat does not favour the parameter model in the primary comparison. Generality beyond this regime — other CPUs, runtimes, quantisation, batch sizes — is untested, and our claim is bounded accordingly.

7 Conclusion

Under a pre-registered matched-budget test, an analytical weight-plus-KV byte-traffic model did not improve held-out prediction or registered selection over parameter count for CPU decode of pruned Llama-3.2-3B. The open question the literature leaves — when KV traffic contributes enough independent variation to change a pruning decision — now has one measured negative point, with the traffic model's over-optimism quantified. The one qualification is localised, not global: excluding the saturated longest context, the traffic model does win the 4-thread contest (Section 4.2), so the practical boundary this study draws is a context boundary as much as a model boundary.

Statement of AI Use

The author uses the AI tools Muse (Muse Spark 1.3, Meta) for design, execution, drafting, and grammatical editing assistance; Perplexity AI for literature-discovery assistance; and Google Gemini and Groq-hosted models for independent review assistance. The author reviews, edits, and attests to the accuracy of all contents.

Data availability

All timing records, the frozen analysis, the paired-block manifest, the pre-registered specification with its amendment record, and the analysis, regret, sensitivity, and bootstrap scripts are archived with the study and accompany the submission as supplementary material.

References

Ziyang Wang, Jiangfeng Xiao, Chuan Xiao, Ruoxiang Li, Rui Mao, and Jianbin Qin. GRASPrune: Global Gating for Budgeted Structured Pruning of Large Language Models. ACL 2026 (arXiv:2604.19398).

Wenqian Zhao, Lancheng Zou, Zixiao Wang, Xufeng Yao, and Bei Yu. HAPE: Hardware-Aware LLM Pruning for Efficient On-Device Inference Optimization. ACM Transactions on Design Automation of Electronic Systems, 30(4), 2025. doi:10.1145/3744244.

Xiteng Yao, Taeho Kim, Hengzhi Pei, Xinle Liu, Kyle Ulrich, Leonard Lausen, Ashish Khetan, Xiang Song, George Karypis, and Martin Herbordt. KernelSight-LM: A Kernel-Level LLM Inference Simulator. arXiv:2606.28565, 2026.

KVPruner: Structural Pruning for Faster and Memory-Efficient Large Language Models. arXiv:2409.11057, 2024; ICASSP 2025.

Efficiently Scaling Transformer Inference. MLSys, 2023.

Zhen Bi, Xueshu Chen, Luoyang Sun, Yuhang Yao, Qing Shen, Jungang Lou, and Cheng Deng. RooflineBench: A Benchmarking Framework for On-Device LLMs via Roofline Analysis. arXiv:2602.11506, 2026.

Understanding Performance Implications of LLM Inference on CPUs. IISWC, 2024.

LLM Inference Unveiled: Survey and Roofline Model Insights. arXiv, 2024.

ZipLM: Inference-Aware Structured Pruning of Language Models. NeurIPS, 2023.